BATCH PROCESSING

Current Code, Logic & Data Flow

MinIO → PySpark → Iceberg → Trino

A code-grounded guide to the batch pipeline

Purpose

Explain what the repository currently implements, how one complete source run moves through the batch code, where outputs are stored, and which parts are not wired into Airflow. This guide describes code behavior; it does not claim that containers are running or that a new execution was performed.

Repository code reviewed: egypt-water-intelligence

Snapshot date: 29 October 2026

1. Executive summary

The batch code takes one completed generator run from MinIO, validates and cleans its NDJSON event files with PySpark, calculates quarter-hour district water balances, writes Silver and Gold Iceberg tables, and validates saved outputs through Spark and Trino.

The important boundary is this: the analytical batch job reads archived files in MinIO. It does not consume Kafka offsets directly. Kafka is upstream in the generator-and-archiver path.

In one sentence

Generator events travel through Kafka and are archived as files in MinIO; build_batch.py reads those files for one run_id, creates Silver and Gold Iceberg tables, and validate_batch.py checks the saved tables.

End-to-end flow

Stage	What happens	Output / evidence
Generate	The generator creates reference data and simulated district-sensor and meter-reading events. Events are published through Kafka.	A run_id and generator artifacts.
Archive	The archiver consumes Kafka topics and saves delivery files in MinIO.	water-raw/<run_id>/artifacts and water-raw/<run_id>/delivery.
Transform	build_batch.py downloads one complete run, validates and deduplicates records, calculates water balance, and builds dimensions, facts, and a daily aggregate.	Iceberg tables in water.analytics and water.gold.
Validate	validate_batch.py checks tables, accounting, references, star schema, daily reconciliation, and Trino visibility.	JSON report; success is status=passed.
Query	Trino exposes Iceberg tables to SQL clients and example analysis queries.	Read-only SQL results; query files do not build tables.

Airflow is a separate orchestration layer. Its current batch option runs handoff_smoke.py, not build_batch.py. The analytical transformation is currently a standalone Spark job.

2. Key terms

Term	Meaning in this project
Batch	A finite dataset is processed as a unit. Here, the unit is one complete run_id.
run_id	Identifier for one generated dataset. It keeps source records and output rows associated with that run.
Kafka	The event broker used before batch processing. The archiver saves Kafka events into MinIO. build_batch.py reads MinIO files, not Kafka topics.
MinIO	S3-compatible object storage. It holds raw run files and Iceberg warehouse objects.
Iceberg	A table format that manages table metadata, snapshots, schemas, and data files. Spark writes through the Iceberg table interface.
PostgreSQL catalog	Stores JDBC Iceberg catalog records so Spark and Trino can resolve the same table names. It does not hold large analytical data files.
Trino	SQL query engine connected to the same Iceberg catalog and MinIO warehouse. It queries tables; it is not the PySpark transformation step.

Term	Meaning in this project
UTC	The standard time basis for event timestamps and interval/date fields. A UTC date can differ from Cairo local date around midnight.

3. Repository file map

File or folder	Current responsibility
batch/jobs/build_batch.py	Main analytical PySpark transformation: source checks, parsing, cleaning, quality accounting, 15-minute balance, Gold dimensions/facts/daily aggregation, and Iceberg writes.
batch/jobs/validate_batch.py	Post-build checks over saved Iceberg tables. It reads with Spark and asks Trino for table row counts.
batch/jobs/handoff_smoke.py	Integration smoke test: checks a run and delivery counts, writes an evidence row to Iceberg, then reads it back with Spark. It is not the full analytical job.
batch/Dockerfile	Builds Spark runtime with Spark 3.5.3, Iceberg runtime and AWS S3 support, PostgreSQL JDBC driver, and MinIO Python client.
docker-compose.yml	Defines services and networking. Spark is under the tools profile; Trino and Iceberg catalog database are under pipeline.
infrastructure/trino/catalog/water.properties	Configures Trino's water catalog to use the JDBC Iceberg catalog and MinIO warehouse.
infrastructure/catalog-db/init.sql	Initializes PostgreSQL catalog structures used by the Iceberg JDBC catalog.
Generator schemas	Define district-sensor and meter-reading event contracts. Batch schemas and validation rules should match.
analysis/gold_analysis.sql	Example read-only Trino SQL analyses over saved tables; it does not perform Spark transformations.
orchestration/dags and orchestration code	Airflow DAG and task runtime. Current batch option is handoff_smoke only.
batch/jobs/test_batch.py	Not present in the checked batch/jobs folder; this snapshot has no such batch fixture-test file.

4. build_batch.py: transformation logic

This is the main analytical job. Its flow is organized in functions: source_manifest, read_events, clean_events, water_balance, dimensional_outputs, write_run, and build.

4.1 Select and verify one run

The command requires a run_id and checks its naming pattern. The MinIO client confirms the run is complete, the generator validation report says passed, and the manifest identifies the same run. If these checks fail, the analytical build stops.

The job uses these source objects:

```
<run_id>/_RUN_COMPLETE
<run_id>/artifacts/run-manifest.json
<run_id>/artifacts/validation-report.json
<run_id>/artifacts/districts.ndjson  and  meters.ndjson
<run_id>/delivery/district-sensors.ndjson  and  meter-readings.ndjson
```

The source fingerprint is computed from object metadata such as ETag, size, last-modified time, and version ID, plus the pipeline version. This is change detection; it is not a full reread-and-hash of every file's content.

4.2 Track the batch and control reruns

water.analytics.batch_manifest records run_id, source fingerprint, target namespaces, pipeline version, status, and update time. The job writes processing before table writes and completed after the outputs are written and the source fingerprint is checked again.

- An unchanged, already completed run is skipped on a normal rerun.
- If the source or output namespace changed after completion, the job requires --reprocess.
- With --reprocess, only that run_id's rows are replaced; other run IDs are preserved.
- Iceberg commits tables separately. batch_manifest is a completion record, not a transaction covering every table. A partial failure should leave the run as processing, and the validator rejects it as incomplete.

4.3 Download and parse NDJSON

The MinIO client downloads the selected run's objects into a temporary directory inside the Spark container. Spark reads the NDJSON using explicit schemas, then temporary files are removed. This is whole-run file processing, not Kafka-offset consumption or checkpointed chunks.

Reference files provide known districts, meters, and attributes. Delivery files contain events. The code checks that meter references point to known districts before cleaning.

4.4 Clean events, deduplicate, preserve rejected evidence

clean_events checks JSON parsing, schema version, selected run_id, event ID, reference membership, allowed quality code, explicit UTC timestamps, expected interval length and alignment, event-time semantics, and finite non-negative measurements. District sensor events are checked against expected inlet and pressure-monitor references.

Invalid deliveries are written to rejected_events with the raw JSON and reason labels. This preserves evidence so a reviewer can inspect why a row did not enter the clean table.

- Identical valid copies of an event are collapsed by run_id and event_id.
- Conflicting valid copies of one event ID stop the build; code does not choose one arbitrarily.
- quality_report records raw_records, accepted_records, rejected_records, and duplicate_records for each stream.

The validator checks this accounting equation for each stream:

```
raw_records = accepted_records + rejected_records + duplicate_records
```

4.5 Calculate the 15-minute water balance

water_balance creates the expected UTC 15-minute timeline for each district. It groups one-minute district sensor observations into each quarter-hour and sums inlet volume. It groups meter readings by district and quarter-hour and sums consumed volume. It joins those values onto the expected district/time grid so missing periods remain visible.

Field / rule	Meaning
inlet_volume_m3	Sum of district sensor inlet volume during the quarter-hour.
metered_consumption_m3	Sum of expected meters' consumption for the district and interval.
inlet_minutes	Count of valid one-minute inlet sensor records; 15 are expected.
expected_meters / meter_count	Expected meter count from references compared with actual valid readings.

Field / rule	Meaning
complete	True only when inlet_minutes is 15 and meter_count equals expected_meters.
gap_m3	For a complete interval: inlet volume minus metered consumption. Null for incomplete intervals.
gap_pct	100 × gap_m3 / inlet_volume_m3 only when complete and inlet volume is positive.

Interpret a gap carefully

A gap is an analytical indicator for investigation, not proof of a leak. Meter coverage, sensor quality, transfers, storage, and other operating conditions can affect the balance.

4.6 Build Gold dimensions, facts, and daily aggregate

dimensional_outputs creates a star-schema-style set of Iceberg tables. Dimensions hold descriptive or calendar attributes; facts hold measurements at declared grains. The daily table joins interval balance to district attributes, then aggregates complete intervals.

Gold table	Grain and purpose
dim_district	One district reference row per run_id and district_id; descriptive district attributes.
dim_meter	One meter reference row per run_id and meter_id; customer and district attributes.
dim_interval	One UTC quarter-hour per run_id and interval_start_utc; date, hour, minute, quarter-of-day, and interval key.
fact_water_balance_15m	One district per 15-minute interval per run_id; balance measures and completeness fields.
fact_meter_consumption_15m	One meter per 15-minute interval per run_id; event and consumption measure.
gold_district_daily_water_balance	One district per UTC date per run_id; daily totals, complete/incomplete counts, coverage, gap, and district descriptors.

Daily aggregation counts all quarter-hour periods for coverage but sums inlet and metered volumes only for complete periods. It calculates gap_pct from daily total volumes; it does not average interval percentages.

5. Outputs and medallion layers

Bronze, Silver, and Gold describe the data's purpose and quality stage. Physical storage uses MinIO buckets and Iceberg namespaces.

Layer	Where it is	Current contents
Bronze / raw	MinIO bucket water-raw under a run_id prefix.	Generator artifacts and archived delivery NDJSON; source read by build_batch.py.
Silver / analytics	Iceberg catalog water, namespace analytics; warehouse objects in MinIO water-warehouse.	Reference snapshots, cleaned events, rejected rows, 15-minute balance, quality_report, batch_manifest.
Gold / analytics-ready	Iceberg catalog water, namespace gold; same configured warehouse.	Dimensions, meter and water-balance facts, and daily district aggregate.

water-raw is not the Iceberg warehouse. Raw files are inputs; Iceberg manages separate tables in the warehouse. Do not manually copy Parquet files into the warehouse path or use Hive partition-repair commands for these Iceberg tables.

6. Iceberg, catalog, and Trino

Component	Role
MinIO water-warehouse	Stores Iceberg table data files and metadata/snapshot files.
PostgreSQL catalog database	Stores catalog records needed to resolve Iceberg namespaces and table metadata; not the large table data files.
Spark Iceberg connector	Creates namespaces and tables, writes format version 2, and overwrites only the selected run_id on rerun.
Trino water catalog	Uses the same JDBC catalog and MinIO warehouse settings so it can query Spark's Iceberg tables.
Trino SQL	Reads tables and calculates results. SQL analysis files do not rebuild tables.

Inside Docker, Spark and Trino reach MinIO at minio:9000. Trino is reached by containers at trino:8080 and from Windows at localhost:18054.

What Iceberg means in this code

Spark writes through the Iceberg table API, not a plain Parquet writer. Iceberg manages commits and table metadata around its data files. The code explicitly sets format-version 2 and partitions new tables by run_id; write_run does not explicitly set a separate file-format property.

7. validate_batch.py: what the checks prove

The validator opens saved tables through Spark, filters to the selected run_id, computes row counts and order-independent checksums, and compares Trino row counts for the same tables. It verifies that batch_manifest is completed for the current source fingerprint and namespaces.

Area	Implemented checks
Keys and grain	Declared keys are non-null and unique.
Source and completion	Fingerprint and namespaces match a completed batch_manifest record.
References and event rules	Meters reference known districts; clean events have valid schema, times, measures, quality codes, and sensor references.
Rejected evidence	Rejected deliveries contain a reason, raw JSON, and an allowed stream label.
Quality accounting	Raw = accepted + rejected + duplicates; report reconciles with clean/rejected tables and generator manifest.
Water balance	Expected grid exists; intervals align to UTC 15-minute boundaries; measures reconcile to clean events; completeness matches coverage.
Gold schema	Dimension and fact keys reconcile; daily Gold values match an independently calculated daily aggregate.
Trino visibility	Trino can query each saved table and returns the same row count Spark observed.

Meaning of status=passed

The checks implemented in this validator passed for this run. This is evidence of table consistency and Trino visibility, but it does not prove every business scenario, a real-world leak, or production-scale behavior.

8. Smoke test versus analytical batch

Job	Purpose and current behavior
build_batch.py	Analytical transformation: reads the run, cleans and deduplicates events, creates Silver and Gold outputs, and marks batch_manifest completed.
validate_batch.py	Analytical post-build verification: checks saved tables in Spark and checks Trino row counts.
handoff_smoke.py	Infrastructure smoke test: verifies run metadata and delivery counts, writes a small row to water.handoff.pipeline_runs, then reads it back through Spark. It does not calculate balances or build Gold facts/dimensions.
Airflow spark_batch	Current allowed batch_job is handoff_smoke only. Airflow separately verifies smoke evidence through Trino; it does not invoke build_batch.py or validate_batch.py.

A successful Airflow smoke run proves the service handoff path works. It does not by itself prove the analytical build completed. For that, use build output plus validate_batch.py status=passed.

9. Runtime and configuration

File	Relevant current settings
batch/Dockerfile	Spark 3.5.3; Iceberg Spark runtime 1.7.1 for Spark 3.5 / Scala 2.12; Iceberg AWS bundle; PostgreSQL JDBC driver; MinIO Python client 7.2.10. spark-submit uses local[2] and 1 GB driver memory.
docker-compose.yml	MinIO minio:9000; catalog database catalog-db:5432; Trino trino:8080 inside Docker and localhost:18054 from Windows. Spark uses tools; Trino and catalog-db use pipeline.
water.properties	Trino Iceberg JDBC catalog; PostgreSQL at catalog-db:5432; warehouse s3://water-warehouse/; MinIO minio:9000; path-style access.
handoff_smoke.py	Creates the shared SparkSession with UTC timezone, Iceberg extension, JDBC catalog, and MinIO S3FileIO settings. build_batch imports this helper.

10. Practical execution sequence

Run these from the repository root in WSL. Replace RUN_ID with the identifier shown under MinIO water-raw. Start dependencies first; rebuild the Spark image after code or Dockerfile changes.

```
docker compose --profile pipeline up -d minio catalog-db trino
docker compose --profile tools build spark
docker compose --profile pipeline --profile tools run --rm spark /opt/spark/work-
dir/jobs/build_batch.py --run-id RUN_ID
docker compose --profile pipeline --profile tools run --rm spark /opt/spark/work-
dir/jobs/validate_batch.py --run-id RUN_ID
```

Expected evidence: build reports status completed (or skipped when unchanged and already completed); validator reports status passed. A job that fails partway must not be described as completed.

Example Gold query

```
SELECT district_id, date_utc, total_periods, complete_periods,
       incomplete_periods, inlet_volume_m3, metered_consumption_m3,
       gap_m3, gap_pct, coverage_pct
FROM water.gold.gold_district_daily_water_balance
WHERE run_id = 'RUN_ID'
ORDER BY date_utc, district_id
```

This query reads a saved Gold Iceberg table through Trino; it does not create or modify the table. Filter by run_id so separate generated runs are not mixed.

11. Current implementation and boundaries

Implemented in checked code	Not wired into analytical Airflow / limits
Run-scoped MinIO input with completion, generator-validation, and fingerprint checks.	Kafka-offset streaming, per-file checkpoints, and incremental micro-batches are not what build_batch.py does.
Schema-aware parsing, validations, deduplication, rejection evidence, and quality counts.	Airflow does not currently run build_batch.py; its batch option launches handoff_smoke.py.
15-minute balance over an expected interval grid with explicit completeness.	A positive gap is not a confirmed leak and does not encode every operating cause.
Silver Iceberg tables and Gold dimensions, facts, and daily district aggregation.	Output tables commit one at a time; batch_manifest is not a transaction across all tables.
Spark checks plus Trino row-count read-back and daily reconciliation.	Trino/Spark validation compares row counts, not every row byte-for-byte.
Example read-only analysis queries in gold_analysis.sql.	The checked batch/jobs folder does not contain test_batch.py.

The word 'incremental' in build_batch.py's docstring needs context: a normal rerun skips unchanged completed input, while --reprocess can replace that run's output. The code does not resume from a mid-file checkpoint.

12. Short explanation for a project review

I built a run-scoped PySpark batch transformation that reads completed event files from MinIO, validates and deduplicates sensor and meter records, preserves rejects with reasons, and calculates complete 15-minute district water balances. It writes cleaned, quality, fact, dimension, and daily aggregate outputs as Iceberg tables in MinIO's warehouse. A separate validator checks table grain, quality accounting, references, balance and daily reconciliation, and Trino row-count visibility. The current Airflow DAG still runs the infrastructure smoke test rather than this analytical build.

13. Glossary and source files

Term	Plain-language meaning
Artifact	Run-level supporting file, such as the manifest, validation report, or reference data.
Delivery	Archived event records that the batch job transforms.
Grain	What one table row represents, such as one district and one 15-minute interval.
Dimension	Descriptive context for filtering or grouping facts, such as district, meter, or time.
Fact	Measured values at a declared grain, such as water balance or meter consumption.
Completeness	Whether all expected sensor minutes and meter readings are present for an interval.
Fingerprint	Change-detection value based on source-object metadata and pipeline version.
UTC	Coordinated Universal Time; the pipeline's standard basis for timestamps and dates.

Files checked: batch/jobs/build_batch.py; batch/jobs/validate_batch.py; batch/jobs/handoff_smoke.py; batch/Dockerfile; docker-compose.yml; infrastructure/trino/catalog/water.properties; infrastructure/catalog-db/init.sql; orchestration/dags/egypt_water_pipeline.py; orchestration/dags/water_pipeline/runtime.py; analysis/gold_analysis.sql; and Generator schemas.

This is a code-state guide, not a fresh run report. For run evidence, attach the actual build output, validator JSON report, and a Trino query result for the same run_id.